

ProbOS — Month 2 Plan

Nisong Monyimba

July 4, 2026

Theme

Sequential inference plus bringing the C++ kernel into the Python package. Month 1 built a forward-simulation kernel: draw priors, propagate, summarise, explain. Month 2's central gap is **inference** — given observed data, update beliefs about latent state and parameters online. That capability is the particle filter, and it is the spine around which every other Month 2 workstream is organised.

Week 5 — Particle Filter Core

Goal: a working sequential Monte Carlo filter, validated against a known analytical case before touching `BatteryModel2Cell`.

- `python/src/particle_filter.py`
 - `ParticleFilter` class: init from prior, `predict()` step (reuses `Model.forward_batch()`), `update()` step (likelihood weighting), `resample()` step (systematic resampling)
 - Effective sample size (ESS) diagnostic, resample-when-ESS-low trigger
- Validate against a linear-Gaussian model with a known Kalman filter solution first, then apply to `BatteryModel2Cell` with synthetic noisy temperature observations.
- Study guide: Chopin & Papaspiliopoulos (2020) Ch 8–10 on resampling schemes; Naesseth et al. (2019) for the unifying SMC algorithm template.

Exit test: filter posterior mean tracks true latent state within theoretical bounds on the linear-Gaussian case; ESS never collapses without triggering resampling.

Week 6 — pybind11: C++ Kernel Enters the Python Package

Goal: stop maintaining two parallel implementations. Bind `BatteryCell` and `MonteCarloEngineOMP` into Python via `pybind11`.

- `cpp/bindings/` — `pybind11` module definition
- `setup.py` / `CMakeLists.txt` updates for building the extension
- Cross-validation test: C++ and Python engines must agree on percentiles for the same seed within floating-point tolerance

Exit test: `import probos.cpp` works from a plain `pytest` run in CI.

Week 7 — FastAPI Service Layer

Goal: expose the kernel over HTTP.

- `POST /simulate` — run `MonteCarloEngine`, return percentiles and convergence summary
- `POST /sensitivity` — run `SobolSensitivity`, return S1/ST indices
- `POST /filter` — run `ParticleFilter` against posted observation data
- Pydantic schemas mirroring the `Model/Distribution` constructor signatures

Exit test: a `curl POST` to `/simulate` returns valid JSON matching a direct Python call to the same `MonteCarloEngine`.

Week 8 — Cross-Discipline Batch 2 + Month 2 Retrospective

Goal: show the particle filter also generalises across domains.

- Port the ED queue model to a filtering setting (infer true arrival rate from observed queue-length time series)
- Port the clinical trial model to sequential monitoring, using the particle filter instead of batch nested-MC
- `docs/retrospectives/month2_retrospective.md` plus `month3_plan.md`

Exit test: both filtering examples pass validation against ground truth, with test coverage matching Month 1's standard.

Standing Month 2 commitments (every week)

- Run `check_ci.sh` after every push, no exceptions
- Update `docs/study/study_guide.md` same-session as any new file needing external reading
- `mypy` strict and `ruff` clean before any commit
- No claimed speedup or result without a genuine benchmark backing it

What was actually accomplished (retrospective)

- **Week 5 – ParticleFilter Core:** bootstrap SIR particle filter, validated against an exact closed-form 1D Kalman filter (implemented directly, not imported) on a linear-Gaussian test case. Genuine mathematical proof, not a smoke test: posterior mean/std converge to the exact solution, with Monte Carlo error provably shrinking as N grows from 100 to 2000.
- **Week 6 – pybind11 bindings:** the Week 4 C++ kernel (`BatteryCell`, `MonteCarloEngineOMP`) bound into the Python package as the `probos.cpp` extension, ending the two-disconnected-implementations problem. Single-step numerical correctness proven to $\text{rtol} \leq 1e-8$ against the Python reference. A known P05–P95 spread discrepancy between the two engines (simplified C++ parameter sampling vs full Python priors) was documented and explicitly tested-for, not silently accepted.

- **Week 7 – FastAPI Service Layer:** `/simulate`, `/sensitivity`, `/filter` endpoints, each verified to match a direct Python kernel call at `rtol=1e-10`. Resource-exhaustion bounds enforced via Pydantic before any request reaches kernel code.
- **Week 8 – Cross-Discipline Batch 2:** `ParticleFilter` demonstrated across two more domains (hospital operations, clinical trials), each validated against an independently computable ground truth. See `docs/monthly_plans/month2/week8/main.tex` for full detail.
- **Comprehensive mid-month hardening pass** (between Weeks 7 and 8, prompted by explicit user request to leave “no stone unturned” before continuing): a systematic, file-by-file audit of the entire repository found and fixed a substantial number of real issues that had accumulated silently, including:
 - `mypy --strict` had never actually been run against `python/tests/` in this project’s history – running it surfaced ~ 150 findings across 3 files (all fixture-typing or deliberate-negative-test patterns, zero real bugs), fixed properly and the CI/audit scope permanently extended to cover it going forward.
 - The full ruff configuration declared in `pyproject.toml` (I, UP, B, SIM rule sets) had never actually been enforced – every invocation throughout the project used a narrower `--select E,F,W` override. Running the full config surfaced 46 findings, including a genuinely tautological test assertion that was rewritten to test its own stated intent rather than papered over.
 - A genuine license mismatch: `README.md` said MIT throughout while the actual `LICENSE` file is Apache-2.0 – a real, standalone factual/legal error, fixed outright.
 - Eight vestigial `sys.path.insert()` hacks removed across `python/src/` and `python/examples/`, predating the project’s editable install and verified genuinely unnecessary via real subprocess import tests (not just AST parsing), both from the repo root and from outside the repo directory.
 - `Model.validate_params()/validate_state()` had docstrings falsely claiming to be called automatically by `MonteCarloEngine` – genuinely wired into `MonteCarloEngine`, `ParticleFilter`, and `SobolSensitivity` for real, adding a genuine early-failure safety check across all three engines.
 - Several smaller off-by-one documentation errors (the `Distribution ABC` has 5 concrete classes, not 4 as `README` claimed; the server registers 4 example models, not 5 as its own docstring claimed) and 3 stale manuscript figure duplicates causing the compiled PDF to embed outdated plots.

This pass is documented in detail across the individual commits from that period; it is summarised here because it materially affected Month 2’s overall quality, not just Week 8’s.

What went right

1. Every new capability (particle filter, pybind11 bindings, FastAPI endpoints, Week 8’s cross-discipline filters) was validated against an independently computable ground truth before being trusted – the same discipline established in Month 1, consistently maintained.
2. Real bugs were found and fixed rather than hidden or rationalised at every stage this month: the prior-mismatch bug in Week 8’s clinical trial validation, the reproducibility bug in the ED

queue ground-truth generator, the false claims in `validate_params`'s docstring, and the license mismatch are all documented honestly in their respective commits.

3. The mid-month comprehensive audit demonstrates the project's own quality-assurance tooling (`pre_week_audit.sh`, `docs/standards/quality_standards.md`) is not just aspirational – when genuinely applied comprehensively (not just the narrower subset previously enforced), it found real, substantive issues, and those enforcement gaps were then permanently closed rather than re-opening next week.

What was harder than expected

1. Designing Week 8's clinical trial filter required real domain reasoning, not mechanical reuse: the naive approach (directly reusing `ClinicalTrialModel.forward_batch()`'s own stochastic simulation inside the standard predict/update loop) would have caused severe weight degeneracy from comparing independently-simulated discrete outcomes against real data. The correct design (deterministic state transitions against known real data, informative likelihood scoring via particle parameters) took real first-principles thinking to arrive at.
2. The mid-month audit revealed that several “we already checked this” assumptions were false – notably, that the full declared ruff/mypy configuration was actually being enforced anywhere. This is a useful, humbling lesson: a checklist that is declared but not actually run provides zero real assurance, regardless of how thorough it looks on paper.

Numbers at Month 2 close

- Python tests: 368/368 passing
- C++ tests: 13/13 passing
- mypy strict: 0 errors (entire `python/` tree, including `python/tests/` – a scope extension made this month)
- ruff: 0 warnings (full declared configuration, not a narrowed subset – a scope extension made this month)
- Coverage: 90.80%
- 0 bandit findings, 0 known dependency CVEs